COMPILER DESIGN

[CS-603]

UNIT-5

CODE OPTIMIZATION

SYLLABUS-

Introduction to Code Optimization: Sources of optimization of basic blocks, loop in flow graphs, dead code elimination, loop optimization, introduction to global data flow analysis, code Improving transformation, Data flow analysis of structure flow graph symbolic debugging of optimized code.

INTRODUCTION TO CODE OPTIMIZATION

Code Optimization is the process of improving a program's code so that it runs faster, uses less memory & consumes fewer resources without changing its output or functionality.

• When a programmar writes a program, the generated code may contain unnecessary instructions.

The compiler improve this code by removing extra operations & making it more efficient.

This process is called code optimization.

Objectives of Code Optimization -

1. Reduce execution time.
2. Reduce memory usage.
3. Improve program efficiency.
4. Minimize unnecessary computations.
5. Generate better machine code.

Example:

Before Optimization:

```
a = b+c;
d = b+c;
```

After Optimization:

```
t = b+c;
a = t;
d = t;
```

Here, b+c is calculated only once, making the program faster.

[VIMP]

SOURCES OF OPTIMIZATION OF BASIC BLOCK

A Basic Block is a sequence of program statements that has only one entry point and one exit point. The compiler performs optimization inside a basis block to make the code faster & more efficient.

The main sources of optimization of a basic block are the techniques used to remove unnecessary operations & improve code quality.

1. Common Subexpression Elimination

If the same expression is calculated more than once & the values of its variables do not change, the compiler computers it only once and reuses the result.

Example:

```
a = b+c;          t = b+c;
d = b+c;          a = t;
                  d = t;
```

Advantage -

- Reduces repeated calculations.
- Saves execution time.

2. Constant Folding

When an expression contains only constant values, the compiler calculates the result during compilation instead of during execution.

Example:-

```
x = 10+20;        x = 30;
```

Advantage-

- Reduces runtime computations
- makes execution faster.

3. Constant Propagation

If a variable has a constant value, the compiler replaces the variable with constant wherever possible.

Example:-

```
a = 10;           a = 10;
b = a+5;          b = 10+5;
```

Advantage-

- Simplifies expressions
- Helps further optimization.

4. Dead Code Elimination

Statements whose results are never used are called dead code. The compiler removes such statements.

Example:-

```
a = 10;        If b is never      →   a = 10;
b = 20;           used later
```

Advantage-

- Reduces code size
- Saves memory & execution time.

5. Copy Propagation

When one variable is assigned to another, the compiler replaces the copied variable with the original variable whenever possible.

Example:-

```
a = b;                    a = b;
c = a + 5;                c = b + 5;
```

Advantage-

- Reduces unnecessary variable usage.
- Simplifies code.

6. Algebraic Simplification

The compiler applies simple mathematical rules to simplify expressions.

Example:-

```
a = b+0;        a = b;
c = d * 1;      c = d;
```

Advantage -

- Remove unnecessary operations.
- Improves execution speed.

7. Strength Reduction

Expensive operations are replaced with equivalent but faster operations.

Example:-

```
a = b * 2;      a = b + b;
```

Advantage -

- Reduces execution cost.
- Improves performance.

Loop In Flow Graph

A loop in a flow graph is a set of nodes (basic blocks) that are executed repeatedly until a specified condition becomes false.

In simple words, a loop is a path in the flow graph where control returns to an earlier node & executes the same statements again & again.

What is Flow Graph?

A Flow Graph is a graphical representation of a program's control flow.

- Each node represents a basic block.
- Edges (arrows) show the flow of control from one block to another.

Loop in a Flow Graph-

A loop exists when:

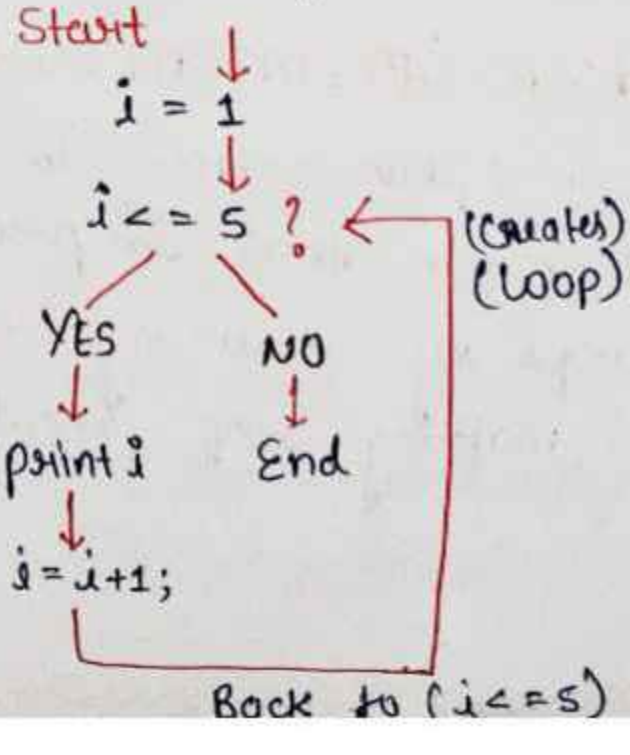
1. There is a header node (entry point of the loop).
2. Control eventually returns to the header node.
3. The statements inside the loop can execute multiple times.

Example:-

Program -

```
i = 1
while (i<=5)
{
  print("%d", i);
  i++;
}
```

Flow Graph -



Importance of loops in Optimization

Since loops execute many times, optimizing them can greatly improve program performance.

The compiler performs:

- Loop invariant code motion
- Strength reduction
- Dead code elimination
- Common subexpression elimination.

Advantages of using loops:-

1. Reduces code repetition.
2. Makes programs shorter & easier to understand
3. Saves memory.
4. Improves program efficiency
5. Allows the compiler to perform optimizations.

Loop OPTIMISATION. [IMP]

Loop optimization is a compiler optimization technique that improves the performance of loops by reducing unnecessary computations & making loop execution faster.

Since loops are executed many times, even a small improvement inside a loop can significantly increase the overall speed of a program.

In Simple words -

A loop may contain operations that are repeated unnecessarily in every iteration. The compiler analyzes the loop & modifies it so that fewer instructions are executed while producing the same output.

This process is called loop Optimization.

Need of loop Optimization -

- To reduce execution time.
- To avoid repeated calculations
- To improve CPU utilization
- To reduce memory access.
- To increase program efficiency.

Common loop Optimization Techniques -

1. Loop Invariant Code motion

If an expression inside a loop gives the same result in every iteration, it is moved outside the loop.

Before Optimization

```
for( i=0; i<10; i++)
{
    x = a + b;
    y = x * i;
}
```

After Optimization

```
x = a+b;
for (i=0; i<10; i++)
{
    y = x * i;
}
```

Benefit:- The expression a+b is calculated only once.

2. Strength Reduction

Expensive operations are replaced with faster operations.

Before:-

```
for(i=0; i<10; i++)
{
    x = i * 2;
}
```

After:-

```
for(i=0; i<10; i++)
{
    x = i + i;
}
```

Benefit:- Addition is generally faster than multiplication.

3. Dead Code Elimination-

Removes statements whose results are never used.

Before:-

```
for(i=0; i<10; i++)
{ a = i;
}
```

After:-

If a is never used later, the statement can be removed.

Benefit:- Reduce unnecessary instructions.

4. Induction Variable Elimination

Removes extra variables that change in a predictable way with the loop counter.

Before:-

```
j=0;
for(i=0; i<10; i++)
  {
    j=j+2;
  }
```

After:-

The compiler may use only one variable instead of maintaining both i and j.

Benefit:- Saves computation time.

Advantages of Loop Optimization -

- Faster execution of loops.
- Reduced CPU time
- Better memory utilization
- Improved overall program performance.
- More efficient generated code.

INTRODUCTION TO GLOBAL DATA FLOW ANALYSIS

Global Data Flow Analysis is a compiler technique used to collect information about how data (variable & expressions) moves throughout the entire program. It helps the compiler understand the flow of values between different basic blocks & perform better code optimization.

Unlike local analysis, which works inside a single basic block, global data flow analysis examines the whole control flow graph of the program.

Easy Explanation:

When a program contains many basic blocks connected by branches & loops, the compiler needs to know:

- Where a variable gets its value.
- Where the value is used.
- Whether a value is still valid.
- Whether some calculations are unnecessary.

To find this information, the compiler performs Global Data Flow Analysis.

Basically, it tracks how data travels through the entire program.

Need for Global Data Flow Analysis -

1. The compiler uses global data flow analysis to
2. Improves code optimization.
3. Remove unnecessary calculations.
4. Detect unused variables.
5. Reduce execution time.
6. Generate efficient machine code.

Example:-

```
B1:
  a = 10;
  b = 20;
B2:
  c = a + b;
B3:
  d = c * 2;
```

Analysis:-

- Value of a is defined in B1.
- Value of b is defined in B1.
- Both value are used in B2.
- Value of c is generated in B2 & used in B3.

The compiler tracks this flow of data across all blocks.

Information Collected by Data Flow Analysis -

1. Reaching Definitions -

Determines which variable definitions can reach a particular point in the program.

```
Ex:- a = 10;
     ....
     x = a;
```

The definition a = 10 reaches the statement x = a.

2. Available Expressions -

Determine expressions that have already been computed & whose values are still available.

```
Ex:- x = a+b;
     y = a+b;
```

The expression a+b is available for reuse.

3. Live Variables

Determines whether a variable's value may be used in the future.

```
Ex:- a = 10;
     print(a);
```

Variable a is live because it will be used later.

4. Dead Variables

Variables whose values will never be used again.

```
Ex:- a = 10;
     a = 20;
```

The first value to becomes dead because it is overwritten.

Types of Data Flow Analysis-

1) Forward Analysis

Information moves in the same directions as program execution.

Example:-

- Reaching Definitions
- Available Expressions.

2) Backward Analysis

Information moves opposite to program execution.

Example:-

Live Variable Analysis.

Applications

1. Common subexpression elimination
2. Constant Propagation.
3. Dead Code Elimination.
4. Loop Optimization.
5. Register Allocation
6. Code motion.

Advantages

- Produces optimized code
- Reduces execution time.
- Improves memory utilization
- Enchances overall program performance.

CODE IMPROVING TRANSFORMATION

Code Improving Transformation are compiler techniques used to modify a program into a more efficient form without changing its output or behaviour.

These transformation improve the speed, memory usage & overall performance of the generated code.

- When a programmer writes a program, the code may contain unnecessary calculations, extra variables or inefficient instructions.

The compiler transforms such code into a better form that executes faster & uses fewer resources.

This process is called Code Improving Transformation.

NOTE:- The transformed code must produce the same output as the original code.

Objectives of Code Improving Transformations -

1. Reduce execution time
2. Reduce memory usage
3. Eliminate unnecessary computations
4. Improves CPU utilization.
5. Generate efficient machine code.

Common Code Improving Transformations-

1) Local Transformation

Local Transformation is an optimization performed within a single basic blocks. The compiler improves the code without considering other basic blocks.

Basically, The compiler looks at only one basic block & removes unnecessary operations inside that block.

Example:-

```
a = b + c;          t = b + c;
d = b + c;          a = t;
                    d = t;
```

Here, optimization is done with the same basic block.

Characteristic:-

- Works on one basic block only.
- Simple & fast.
- Does not analyze the entire program.

Ex:- Common subexpression Elimination, Constant ~~flo~~ folding.

4) Global Transformation

Global Transformation is an optimization performed across multiple basic blocks using the Control Flow Graph (CFG).

Basically, the compiler analyzes the whole program & improves code by considering the flow of data between different basic blocks.

Example:-

```
B1:
  a = 10;
B2:
  b = a + 5;
B3:
  c = a + 10;
```

The compiler knows that the value of a defined in B1 is used to in B2 and B3. and performs optimization across blocks.

Characteristics:-

- Work on the entire control flow graph.
- More complex than local transformation.
- Produces better optimization.

Example:- Global common subexpression Elimination
Constant Propagation,
Loop Optimization.

SYMBOLIC DEBUGGING OF OPTIMIZED CODE

Symbolic Debugging is the process of debugging optimized code using symbolic information such as variable names & line numbers.

Need-

During optimization:

- Variables may be removed.
- Statements may be reordered
- Some code may disappear.

This makes debugging difficult.

Functions:-

1. Maps machine instructions to source code.
2. Tracks variables after optimization.
3. Helps locate errors.
4. Supports breakpoints & variable inspections.

Advantage-

- Easier debugging.
- Better error detection.
- Maintains relation between optimized code & source code.